

自由研究アシストアプリ 開発進捗・技術構成まとめ

アプリの裏側で何が起こっているかの共有用

対象: ゼミ1日限定イベント用アプリ

作成日: 2026年6月

1. このアプリについて

まず全体像

小学生が自由研究のテーマを考える手助けをするWebアプリです。カテゴリ(生き物・化学・物理・歴史・ITなど)を選び、AIとチャット形式でやり取りしながらテーマを決め、決まったテーマを保存していく、という流れで使います。ゼミで実施する1日限定のイベント用として開発しているため、長期運用は前提にしておらず、「その日きちんと動けば成功」という設計です。

使用している技術(全体)

役割	使用技術	ひとこと言うと
画面(フロントエンド)	React + Vite	子どもが実際に触る画面を作る部分
サーバー(バックエンド)	Express (Node.js)	画面とAI・DBの間を取り持つ「窓口」
データベース	PostgreSQL(Render無料プラン)	保存したテーマなどのデータを置いておく場所
AI	Claude API(Anthropic)	子どもとの会話・テーマ提案を担当
DB確認ツール	DBeaver	保存されたデータを目で見て確認するツール
デプロイ(公開)	Vercel(フロント) / Render(バックエンド)	作ったものをインターネット上に公開する場所
エディタ	VSCode	コードを書くソフト
バージョン管理	GitHub + GitHub Desktop	変更履歴の保存と自動デプロイのきっかけ

補足:「Vercelでバックエンドも動かしている」というより、実態は**フロントエンドはVercel、バックエンド(Express)とDBはRender**という2社にまたがる構成です。詳しくは3章で説明します。

全体の関係図(ざっくり)



開発・公開の流れ

- ① **コードを書く** — VSCodeでフロント(React)とバック(Express)のコードを編集
- ↓
- ② **GitHubへ反映** — GitHub Desktopで変更を確定し、GitHubのリポジトリにPush
- ↓
- ③ **自動デプロイ** — VercelとRenderがリポジトリの更新を検知し、それぞれ自動でビルド・公開
- ↓

④ 本番反映 — 数十秒～数分後に、公開中のURLに変更が反映される

※ Push後すぐには反映されません。ビルドに少し時間がかかります。

2. ユーザー(子ども)が実際に触る流れ

画面遷移と、その裏で動いていること

画面1 ユーザー番号入力画面

最初に番号(ID)を入力してもらう。これが後でその子のテーマを判別するためのキーになる。



画面2 タイトル画面

「辞書を見る」か「テーマを考える(チャット)」かを選ぶ。



画面3 カテゴリ選択画面

生き物・化学・物理・歴史・IT・社会・生活・自然 の8カテゴリから1つ選ぶ。

↓ (「テーマを考える」を選んだ場合)

画面4 チャット画面

AI(Claude)が「どんなことが気になってる?」と話しかけ、子どもと対話しながらテーマのヒントを出していく。

△ ここでAnthropicのAI APIに通信が発生するため、数秒の応答待ちが発生する。



画面4内 テーマ保存エリア

会話の中で決まったテーマを入力し「保存」ボタンを押すと、サーバー経由でデータベースに書き込まれる。複数個保存できる。



画面5 テーマ一覧画面

これまで保存したテーマを振り返って確認できる画面。

辞書機能(別ルート): カテゴリ選択後、AIを使わずあらかじめ用意した用語集(例: 「光合成」 「酸化」 「摩擦」 など)を見られる画面。AI応答待ちが発生しないので、サクサク使える対比ルートとして用意されている。

3. 裏側の仕組み(技術詳細)

3-1. ディレクトリ構成

リポジトリの中身は大きく「フロント(`src/`)」「バックエンド(`server/`)」の2つに分かれています。

```
jiyukenyu-app_ver2.1/
├─ package.json           # プロジェクト全体の設定・依存ライブラリー一覧
├─ vite.config.js         # フロントエンドのビルド設定
├─ .gitignore             # GitHubに上げないファイルの指定(.envなど)
├─ README.md
├─
├─ public/               # 公開用の静的ファイル
│  └─ icons.svg
├─
├─ src/                  # フロントエンド(画面側)のソース
│  ├─ App.jsx            # 全体の司令塔。どの画面を表示するかを管理
│  ├─ App.css            # 画面のデザイン(CSS)
│  │
│  └─ components/        # 画面パーツごとのファイル
│     ├─ TitleScreen.jsx  # タイトル画面
│     ├─ CategorySelect.jsx # カテゴリ選択画面
│     ├─ ChatBox.jsx      # チャットUI本体
│     ├─ Message.jsx      # チャット1メッセージの吹き出し
│     ├─ DictScreen.jsx   # 辞書画面
│     ├─ SaveThemeArea.jsx # テーマ保存欄
│     ├─ UserIdScreen.jsx  # ユーザー番号入力画面
│     └─ ThemeListScreen.jsx # 保存済みテーマ一覧画面
│
│  └─ services/
│     └─ Claudeapi.js     # フロントからバックエンドのAPIを呼ぶ処理
│
│  └─ data/
│     └─ categories.js    # 8カテゴリの定義(名前・アイコンなど)
├─
└─ server/              # バックエンド(Express サーバー)のソース
   ├─ index.js           # API窓口・Claude API連携・DB接続をすべて担当
   └─ .env               # APIキー等の秘密情報(リポジトリには含めない)
```

※ `src/` (フロント)と `server/` (バック)が同じリポジトリに同居している構成。 `node_modules/` や `dist/` は自動生成されるためGit管理対象外。

3-2. フロントエンド(React + Vite)

子どもが実際に見る画面そのもの。画面は「今どの画面を表示しているか」という状態を1つの変数(`screen`)で管理しており、ボタンを押すとその値が切り替わって画面が変化する仕組み(SPA=シングルページアプリケーション)になっています。

司令塔は `App.jsx`。ここから `components/` 配下のパーツを呼び出して画面を組み立てています。AIへの通信は `services/Claudeapi.js` が窓口となり、バックエンドへの問い合わせを担当しています。

3-3. バックエンド(Express)

フロントエンドとAI・DBの「間」に立つ窓口役のサーバーです。コードは `server/index.js` の1ファイルに集約されており、主なAPI(窓口の種類)は3つです。

エンドポイント	役割
POST /api/chat	子どもの発言とカテゴリ情報を受け取り、Claude APIに転送して返事をもらい、フロントに返す
POST /api/save-theme	確定したテーマをDBの <code>themes</code> テーブルに保存する
GET /api/themes/:userId	そのユーザー番号で保存された過去のテーマを一覧取得する

3-4. Claude API連携の仕組み

カテゴリごとに異なる「指示文(システムプロンプト)」をAIに渡しており、AIが子どもの興味に合わせた問いかけをするよう調整されています。共通ルールとして「答えを直接教えず、問いかけで考えさせる」「やさしい言葉」「3文以内」という指示が土台にあり、その上にカテゴリ別の指示(生き物なら生き物関連の話題に絞る、など)が重ねられます。

3-5. データベースの中身

目 先に知っておくと読みやすい用語

データベース(DB): たくさんの情報を整理して保存しておくための場所。Excelの大きいバージョンのようなイメージ。

テーブル: DBの中にある「表」のこと。1つのDBの中に複数のテーブルを作れる。今回のアプリでは `themes` という名前のテーブル1つだけを使っている。

カラム(属性): テーブルの「縦の列」のこと。何の情報を入れるかを決めた項目名。Excelで言う一番上の見出し行に近い。

レコード(行): テーブルの「横の1行」のこと。1件分のデータがレコードにあたる。テーマを1つ保存するごとに1レコード増えていく。

`themes` テーブルのカラム(属性)は以下の5つです。

カラム名	入っているもの
id	保存ごとの自動採番ID(1, 2, 3, ...と勝手に増えていく通し番号)
user_id	最初に入力したユーザー番号
category	選んだカテゴリ(生き物・化学など)
theme	子どもが決めたテーマの文章
created_at	保存した日時

このテーブルをDBeaverというツールで直接覗いて、ちゃんと保存されているかを確認しながら開発しています。

3-6. データが保存される流れ・取り出される流れ

目 先に知っておくと読みやすい用語

SQL(エスキューエル): データベースに対して「これを保存して」「これを取ってきて」と命令するための専用の言葉。バックエンド(Express)がこの命令文を組み立ててDBに送る。

INSERT(インサート): SQLの命令の1つ。「新しいデータを入れる」という意味。

SELECT(セレクト): SQLの命令の1つ。「データを取り出してくる」という意味。

保存するとき(子どもが「保存」を押した瞬間)

- 1 子どもがテーマを入力して「保存」ボタンを押す
- ↓
- 2 フロントエンドが、バックエンドの `/api/save-theme` という窓口にデータ(ユーザー番号・カテゴリ・テーマ文)を送る
- ↓
- 3 Expressサーバーが受け取って、SQLという命令文(`INSERT`)を組み立てる
- ↓
- 4 PostgreSQL DB が命令を受け、`themes` テーブルに新しい行が1つ追加される
- ↓
- 5 DBから「保存できたよ」の返答 → Expressが「成功」をフロントに返す → 画面に「保存したよ!」と表示

取り出すとき(テーマー一覧画面を開いた瞬間)

逆向きに同じ道を辿るイメージ。

- 1 テーマー一覧画面を開く
- ↓
- 2 フロントエンドが、バックエンドの `/api/themes/:userId` という窓口にユーザー番号を伝える



3 ExpressサーバーがSQL(`SELECT`)を組み立て、「user_id が ○○ のレコードを全部ちょうだい」とDBに依頼



4 DBが該当する行を全部まとめて返す



5 Expressがフロントに渡し、フロントが画面に一覧として表示

つまり、フロント・バックエンド・DBの3者は、保存と取得でまったく同じ道を逆向きに通っているだけと捉えると分かりやすいです。

3-7. ユーザー番号と画面の更新(SessionStorageの話)

ここは知っていると当日とかに役立つ

目 先に知っておくと読みやすい用語

SessionStorage(セッションストレージ): ブラウザの中に用意されている、ちょっとした一時メモ帳のような保存場所。同じタブを開いている間だけ情報を覚えていてくれる。タブを閉じると消える。

※ DBがサーバー側の倉庫だとすれば、SessionStorageは「自分のブラウザのタブ内だけにある付箋」のようなイメージ。

子どもが最初に入力したユーザー番号は、SessionStorageに記録しています。これによって便利な性質が2つ生まれます。

① 画面を間違えてリロードしても、最初からやり直さなくていい

もし子どもがブラウザの更新ボタンを押してしまっても、SessionStorageにユーザー番号が残っているため、再びユーザー番号入力画面に戻されることはありません。途中の続きから使い続けられます。

② タブを閉じてしまっても、データ自体は消えない

SessionStorageはタブを閉じると中身が消えます。でも、ここまで保存したテーマは「ブラウザ側ではなくDB側」に `user_id` と一緒に保存されているため、データそのものは無事です。

もしタブを閉じてしまった子がいても、もう一度アプリを開いて最初の番号入力画面で同じユーザー番号を入力すれば、過去のテーマが `themes` テーブルから読み出されて、また続きから見られます。

整理: データの居場所は2層になっています。

- **ブラウザのSessionStorage** — ユーザー番号などの一時的な情報。タブを閉じると消える、軽快な短期メモ。
- **DB(themesテーブル)** — 保存したテーマ本体。タブを閉じてでも消えない、長期保管庫。

SessionStorageが消えても、ユーザー番号さえ覚えていればDBから復活できる、という設計です。

3-8. デプロイ構成の実態

フロントエンドとバックエンドが別のサービスに分かれて公開されている点が、構成として少し複雑なポイントです。

部分	公開先	役割
フロントエンド(React)	Vercel	子どもが開くURLそのもの
バックエンド(Express)	Render(無料プラン)	API・DB接続を担当する裏方サーバー
データベース	Render(無料プラン)	テーマデータの保存先

GitHubにPushすると、VercelとRenderそれぞれが自動的に最新版を取得してビルド・公開する仕組みになっているため、デプロイ作業自体は手間がかかりません。

4. 補足: 知っておくと役に立つかもしれないこと

いずれも大きな問題ではないが、性質として知っておくと便利な事項

大して役に立たない

以下はアプリの仕組み上「そういうもの」として起こり得る現象です。特別な対応が必要な深刻なリスクというわけではありません。

- Render無料プランのスリープ:** しばらくアクセスがないとサーバーが休止し、再アクセス時に起動で数十秒待つことがある。当日は事前起こしておく予定。
- AI応答の通信時間:** Claude APIへの通信のため、送信から返答まで数秒のタイムラグがある(これは仕様)。
- デプロイ反映の時間差:** GitHubへのPush後、本番URLに反映されるまで数十秒～数分のビルド時間がかかる。

5. 用語ミニ辞典

この資料に出てきた用語の解説

Webアプリの基本

用語	説明
Webアプリ	ブラウザ(Chrome、Safariなど)から開いて使う形式のアプリのこと。スマホやPCにインストールせず、URLにアクセスするだけで使える。
ブラウザ	Webサイトやアプリを表示するソフト。Chrome、Safari、Firefox、Edgeなど。
URL	「https://～」で始まる、Web上のページ住所のこと。
フロントエンド	ユーザーが目で見えて触る画面の部分。ボタン、文字、レイアウトなど、見える側すべて。
バックエンド	画面の裏側で動いているサーバーの部分。データを保存したり、AIに問い合わせたりといった見えない作業を担当する。
サーバー	常時インターネットに繋がっていて、リクエストを受けて答えを返す役割のコンピュータ。
クライアント	サーバーに対してお願いをする側。今回の場合はブラウザがクライアント。
SPA	Single Page Application の略。ページ全体を切り替えず、同じ画面の中身だけを書き換える方式のWebアプリ。動きが軽快。

通信・API関係

用語	説明
API	プログラム同士が情報をやり取りするための「窓口」のこと。「これお願いします」と頼むと「はい、これです」と返事が返ってくる仕組み。
エンドポイント	APIの個々の窓口の名前。1つのサーバーに用途別に複数の窓口を作れる(<code>/api/chat</code> 、 <code>/api/save-theme</code> など)。
リクエスト	クライアント(ブラウザ)からサーバーへの「お願い」のこと。
レスポンス	サーバーからクライアントへの「返事」のこと。
HTTP通信	Web上で情報をやり取りする際の共通ルール(言語のようなもの)。
JSON	データを整理して受け渡すための書式の1つ。プログラム間でやり取りする情報をこの形に整えて送る。
POST / GET	リクエストの種類。POSTは「データを送って何かしてもらう時」、GETは「データを取ってくる時」に使う。

データベース関係

用語	説明
データベース(DB)	たくさんの情報を整理して保存しておくための場所。Excelのとても大きい・しっかりしたバージョンのようなもの。
PostgreSQL	世界中で広く使われているデータベースソフトの1つ。本アプリではRender社の無料プランで運用している。
テーブル	DBの中にある「表」のこと。1つのDBの中に複数のテーブルを作れる。今回は <code>themes</code> という名前のテーブル1つを

	使用。
カラム(列・属性)	テーブルの「縦の列」のこと。「何の情報を入れるか」を決めた項目名。Excelの一番上の見出し行に近い。
レコード(行)	テーブルの「横の1行」のこと。1件分のデータがレコードにあたる。
SQL(エスキューエル)	データベースに「これを保存して」「これを取ってきて」と命令するための専用の言葉。
INSERT	SQLの命令の1つ。「新しいデータを入れる」という意味。
SELECT	SQLの命令の1つ。「データを取り出してくる」という意味。
DBeaver	データベースの中身を目で見て確認できるソフト。開発中、テーブルにちゃんとデータが入っているかをこれで確認している。

ブラウザ側の保存場所

用語	説明
SessionStorage	ブラウザの中に用意されている一時的な保存場所。同じタブを開いている間だけ情報を覚えていてくれる。タブを閉じると消える。今回はユーザー番号の保持に使っている。
LocalStorage	SessionStorageの長期保存版。タブを閉じても消えない。今回のアプリでは使っていない。

開発・公開関係

用語	説明
リポジトリ	プログラムのソースコードをまとめて置いておく場所。GitHub上に作られる。
GitHub	世界中の開発者がリポジトリを置いて使うサービス。コードの変更履歴をすべて記録できる。
Push(プッシュ)	自分のPCで書いた変更をGitHubのリポジトリに送り込む操作。
デプロイ	作ったアプリをインターネット上に公開すること。
ビルド	書いたコードを、ブラウザで動く形に組み立てる工程。
CI/CD	コードを更新するだけで、ビルドから公開までを自動でやってくれる仕組み。今回はGitHubへのPushがきっかけでVercelとRenderが自動的にビルド・公開してくれる。
Vercel	フロントエンドを簡単に公開できるサービス。今回は子どもが触るReact画面の公開に使用。
Render	バックエンドサーバーやDBを公開できるサービス。今回はExpressサーバーとPostgreSQL DBに使用。
スリープ	サーバーが使われていない間に休止状態になること。無料プラン特有の挙動で、再アクセス時に起動時間がかかる。

このアプリに登場する主な技術

用語	説明
React	画面を作るための仕組みの1つ。「コンポーネント」という小さなパーツを組み合わせて画面を構成する。
Vite	Reactで作ったコードを動かしたりビルドしたりするためのツール。
Node.js	本来ブラウザ上でしか動かなかったJavaScriptを、サーバー側でも動かせるようにする実行環境。
Express	Node.js上でサーバーを作るためのフレームワーク。API窓口の用意などを簡単に書ける。
Claude API	Anthropic社のAI「Claude」と通信するための窓口。今回はAIに「子どもの相談に乗ってもらう」役を担当してもらっている。
VSCode	Microsoft製のコードを書くソフト(エディタ)。開発で広く使われている。